Leader election on a unidirectional ring: Algorithm B of Dolev, Klawe and Rodeh

Roman Volochai

Distributed Systems, talk of 4 November 2026

Code, slides and this report:
`https://github.com/RomanVolochai/distributed-systems-unidir-leader-finder-b`

This report accompanies the talk on Section 3 of D. Dolev, M. Klawe, M. Rodeh, *An $O(n \log n)$ unidirectional distributed algorithm for extrema finding in a circle*, J. Algorithms 3 (1982), 245–260. It follows the order of the talk, fills in what the talk had no time for, and gives a map of the proofs with the numbering of the paper. Section 6 collects the points that are easy to misunderstand. Throughout, log is the binary logarithm.

1 Problem

n processes form a ring. Every process sends only to its right neighbour and receives only from its left neighbour. The system is **asynchronous**: delays are arbitrary but finite, every link is **FIFO**, and a process handles its messages one by one. Ids are distinct, all processes run the same program, and n is not known. The task is to find the maximum id; its owner becomes the leader. The cost is the number of messages in the worst case. The lower bound is $\Omega(n \log n)$ (Burns, 1980). Hirschberg and Sinclair conjectured $\Theta(n^2)$ for one-way rings; the simple Algorithm A of the paper (found independently by Peterson) refutes it with $2n \log n + n$, and Algorithm B lowers the constant to 1.5.

2 Idea: from Algorithm A to Algorithm B

Algorithm A (Section 2). Because links are FIFO, a run can be seen as a sequence of *phases*. In a phase the active processes form a smaller ring; the passive ones only relay. For three consecutive active processes $u \to v \to w$, v cannot look to the right, so w decides: v's M1 brings $\max(v)$, and v forwards $\max(u)$ in an M2. If $\max(v)$ is larger than both neighbours (a local maximum), w takes it over. At most half of the values are local maxima, so there are at most $\log n + 1$ phases, and every phase puts one M1 and one M2 on every link: $2n \log n + n$ messages.

Goal. M1 is needed on every link (each active process must announce its value). Algorithm B keeps it and sends M2 over only about half of the links: $n \log n + \frac{1}{2} n \log n$. The paper gets there with three changes applied to A one after another (Section 3.1).

Change 1: only a winner sends M2. v already knows whether $\max(u) > \max(v)$ when u's M1 arrives. (The paper first notes that v could send one bit instead of a number; that saves bits, not messages.) If $\max(u) > \max(v)$, w drops out whatever $\max(w)$ is, so v sends nothing. If $\max(v)$ wins, v becomes passive and sends M2 with *its own* value. The comparison $\max(v)$ vs $\max(w)$ is made by w on v's M1: if $\max(v)$ is bigger, w stores it and enters the new state WAITING. A waiting process that receives M2 becomes active in the next phase; if instead the next M1 arrives, FIFO guarantees that no M2 is coming, and it becomes passive ("two M1s in a row"). The survivors are still exactly the local maxima; the paper remarks that this change alone does not reach $\frac{3}{2} n \log n$.

Change 2: passive processes drop hopeless M2s. Passive processes keep $\max(x)$, the largest value that passed them in an M1. If u and v are neighbouring active processes and a passive x between them has $\max(x) > \max(u)$, then $\max(u)$, which v would represent if it survived, is not the maximum, so x does not forward u's M2. The M1 must still pass, because v needs it to decide about its own value. This is why B sends $(2, \max(v))$ instead of A's $(2, i)$: the passive process can compare.

Change 3: a counter 2^p. After a few phases the active processes may sit on a short arc; then one M2 still crosses almost the whole ring. The M1 of phase p therefore carries a counter 2^p, decreased by passive and waiting processes. The process that receives counter 1 (at distance exactly 2^p) and a value $\geq$ its max becomes waiting itself, a "virtual candidate"; after it the counter is 0 and the M1 is a plain relay. Waiting processes never forward M2, so an M2 of phase p stops within 2^p. The paper stresses that this change alters the behaviour: passive processes can become active again, and survivors need not be local maxima, so the proof for A no longer applies.

Algorithm B (Section 3.2). State: *phase*, *state*, *id*, max (initially *id*); messages M1 $= (1, i, p, c)$ and M2 $= (2, j)$.

from	to	when	sends
active (B1)	waiting	M1 with $i >$ max; max $:= i$	—
active (B1)	passive	M1 with $i <$ max	M2(max)
waiting (B2)	active	M2 arrives; *phase*++	M1(max, *phase*, 2^{phase})
waiting (B2)	passive	M1 arrives first; then as B3	—
passive (B3)	waiting	M1 with $c = 1$ and $i \geq$ max	the M1 with $c = 0$
passive (B3)	passive	other M1: store i if $i \geq$ max and $c \geq 1$; $c{-}{-}$	relay
passive (B3)	passive	M2 with $j <$ max: drop; otherwise forward	—
any	leader	M1 with $i = id$	announcement

3 Correctness: map of Section 3.3

Notation: $A(p)$ is the set of processes that start phase p as active (send an M1 with phase p); p-max(v) is their value at that moment; origin(j) is the process with id j; $d(u, v)$ is the distance from u to v going right. Every process receives exactly one M1 of each phase, which makes phases well defined even though nothing is synchronised.

- **3.3.1 (distinct values).** Induction: M2s of different initiators carry different values, and each activates at most one process, which takes exactly that value.
- **3.3.2 (short M2s).** An M2 of phase p travels at most 2^p links. Let v be the next active process after the initiator u. If $d(u, v) > 2^p$, the process at distance 2^p is waiting and stops it. Otherwise v is waiting (if $\max(v) < \max(u)$) or has a bigger max and drops it.
- **3.3.3.** In one phase each link carries at most one M1 and one M2.
- **3.3.4–3.3.5 (the maximum survives).** The M1 carrying the maximum M makes the next active process, or the process at distance 2^p, wait for M. The holder of M is beaten by nobody on its left, so it sends M2(M); no process can drop it, since nobody has seen a larger value; by FIFO it reaches the waiting process before any M1 of the next phase. Hence every $A(p)$ contains a process holding M.
- **Termination and a unique leader.** By Corollary 3.3.7 below, after at most $\lfloor \log n \rfloor$ phases one active process is left, and by 3.3.5 it holds M. Its M1 goes around to origin(M), which receives its own id. The test cannot fire earlier: by 3.3.6(b) the M1 with value j starts after origin(j) and must pass every other process to come back, and active processes never forward an M1. Unlike in A, the leader is the *owner* of the maximum; our implementation then sends one announcement around the ring (n more messages) so that everybody learns the result.

Lemma (3.3.6, the timetable)**.** *Let $v \in A(p)$ and $j = p\text{-max}(v)$. Then (a) $d(\text{origin}(j), v) = 2^p - 1$; (b) every process y from $\text{origin}(j)$ to v has $p\text{-max}(y) = j$; (c) $d(w, v) \geq 2^p$ for every other $w \in A(p)$.*

Proof map (induction on p). Let u be the nearest process of $A(p-1)$ to the left of v. Then v was activated by u's M2, so j was u's value and $d(u, v) \leq 2^{p-1}$ by 3.3.2. Also $d(u, v) \geq 2^{p-1}$: if $v \in A(p-1)$, this is (c) for $p-1$; otherwise v could only have started waiting through the counter, at exactly 2^{p-1}. So the value moves *exactly* 2^{p-1} in phase $p-1$, and $(2^{p-1} - 1) + 2^{p-1} = 2^p - 1$ gives (a). For (b), the processes between u and v forwarded u's M2, so they had seen nothing larger and stored j; the processes from $\text{origin}(j)$ to u received a smaller value in phase $p-1$ (the M1 of the previous active process, which lost to j). Part (c) follows: an active process closer than 2^p would lie in the segment of (b) and hold j, contradicting 3.3.1.

Why few processes are active, intuitively. Each surviving value "owns" the stretch of the ring from its owner to its holder: all processes on it store this value as max. The stretches of different values are disjoint, and their length doubles every phase. So **Corollary 3.3.7**: $|A(p)| \leq n/2^p$, hence at most $\log n + 1$ phases. Note that the number of actives need *not* halve in every single phase. On the ring $7, 3, 4, 5, 8, 2, 6, 1$ it goes $8 \to 3 \to 2 \to 1$: in phase 1 the value 7 survives although its active neighbour holds 8, because the virtual candidate at distance 2 catches it first. The bound $n/2^p$ still holds ($8/4 = 2$), and that is all the analysis needs.

4 Complexity: map of the message count

Lemma (3.3.8, silence after every run)**.** *Let u send an M2 in phase p, let f be the last process that forwards it, and let g satisfy $d(f, g) = 2^p - 1$. Then no process strictly after f up to g sends an M2 in phase p.*

Proof map. Only active processes start M2s. Let w be the next process of $A(p)$ after u. If $p\text{-max}(w) < p\text{-max}(u)$, w waits and starts nothing, and the active process after w is at distance $\geq 2^{p+1}$ from u by 3.3.6(c), beyond g. If $p\text{-max}(w) = j > p\text{-max}(u)$, then $\text{origin}(j)$ already holds j and drops u's M2, so f is before $\text{origin}(j)$; by 3.3.6(a) $d(\text{origin}(j), w) = 2^p - 1$, so w lies beyond g.

Corollary 3.3.9, step by step. The processes that send M2 in phase p form runs of length $\leq 2^p$ (3.3.2), each followed by $\geq 2^p - 1$ silent processes (3.3.8). In every "run + silence" block the fraction of senders is at most $2^p/(2^p + 2^p - 1)$, and every process sends at most one M2 per phase, so

$$\#\mathsf{M2}_p \;\leq\; n\,\frac{2^p}{2^{p+1} - 1} \;=\; \frac{n}{2} + \frac{n}{2(2^{p+1} - 1)},$$

where the equality is checked by putting the right side over a common denominator. The fraction is $1, \frac{2}{3}, \frac{4}{7}, \frac{8}{15}, \cdots \to \frac{1}{2}$: from phase 1 on, about half of the processes send M2.

Theorem 3.3.10. There is no M2 in the last phase, so at most $\log n$ phases contribute. Summing,

$$\#\mathsf{M2} \;\leq\; \frac{n}{2}\log n \;+\; \frac{n}{2}\Big(1 + \tfrac{1}{3} + \tfrac{1}{7} + \tfrac{1}{15} + \cdots\Big) \;\approx\; \tfrac{1}{2}n\log n + 0.80\,n \;<\; \tfrac{1}{2}n\log n + \tfrac{5}{3}n.$$

The series converges because its denominators double; the paper's $\frac{5}{3}n$ is a generous bound for it. With at most $n(\log n + 1)$ M1s (one per link per phase) the total is $< \frac{3}{2}n\log n + \frac{8}{3}n$. The paper also notes the price in *bits*: every M1 now carries a $\log n$-bit counter, so measured in bits B is worse than an optimised Algorithm A.

5 Implementation and measurements

The Go code is in the folder `code/` of the repository (`https://github.com/RomanVolochai/distributed-systems-unidir-leader-finder-b`). It follows the course guidelines for asynchronous algorithms:

- **one goroutine per process** (`node.run` in `network.go`); links are channels with buffer 1, and between steps a goroutine only stores arriving messages in its FIFO `inbox`;
- **the state is a separate type** (`State`, file `state.go`); **one message is handled by one pure function**: `handleB(State, Message)` returns the new state and the messages to send (`algorithm_b.go`), with step B3 in `passiveB` and the wake-up in `startB`; `node.step` loads the state, takes one message, calls the handler, sends and saves;
- **random order**: the scheduler in `Run` picks, with a seeded generator, a process that has not woken up yet or has a message waiting, and lets it take one step; the global `Delay` (`-delay`) is waited before every step;
- **logging**: every step prints the handled message and the new state, then lines like `P3 sent M2(value=6) to P4`; the summary gives the leader, phases, M1 and M2 counts and the bound;
- **tests** (`dkr_test.go`): the right leader for $n = 1, \dots, 70$ and four kinds of rings, the bound of Theorem 3.3.10, Lemma 3.3.6(a,c), Corollaries 3.3.7 and 3.3.9, and that the schedule does not change any count;
- Algorithm A (`algorithm_a.go`, flag `-algo A`) for comparison.

The Algorithm B in `distributed-framework` runs in the synchronous model and always starts the counter at 1 instead of 2^p, so it loses change 3; this implementation is written anew.

Scaling with n (`go run . -bench -seed 7`; full table in `report/bench.txt`). Election messages M1 + M2, without the n announcement messages. *Random*: average over 20 random rings. *Bit-reversal*: ids in bit-reversed order of positions, e.g. $1, 5, 3, 7, 2, 6, 4, 8$ for $n = 8$; every other value is a local maximum in every phase, so exactly half survive and there are exactly $\log n + 1$ phases (the worst case).

	random (average)				bit-reversal			
n	phases	B	$\frac{\text{B}}{n \log n}$	$\frac{\text{A}}{n \log n}$	phases	B	$\frac{\text{B}}{n \log n}$	$\frac{\text{A}}{n \log n}$
16	4.0	75	1.170	1.700	5	97	1.516	2.250
64	5.3	408	1.062	1.433	7	577	1.503	2.167
256	7.2	2204	1.076	1.400	9	3073	1.500	2.125
1024	9.1	10952	1.070	1.350	11	15361	1.500	2.100
4096	10.8	51867	1.055	1.375	13	73729	1.500	2.083

On bit-reversal rings B sends exactly $\frac{3}{2} n \log n + 1$ messages (n M1 and $n/2$ M2 per phase, plus one M1 in the last phase) and A exactly $2n \log n + n$, so both leading constants are attained; only the $\frac{8}{3} n$ term of the theorem is loose. On random rings the number of phases grows like $\log n$ and both algorithms stay well below their worst case. On sorted rings (`-layout asc/desc`) both need only two phases.

What each change buys ($n = 1024$; *crowded*: ids $1, \dots, 1008$ in order followed by the 16 largest ids in bit-reversal order, so the active processes end up on a short arc).

	bit-reversal		crowded		random (avg. 30)	
	M2	total	M2	total	M2	total
A	10240	21504	4096	9216	6315	13653
A + 1	5120	16384	2048	7168	3267	10606
A + 1 + 2	5120	16384	2048	7168	2790	10129
B = A + 1 + 2 + 3	5120	15361	1040	6145	2002	11057

Change 1 halves M2 everywhere; change 2 helps when values are mixed; change 3 halves M2 again on the crowded ring. On random rings the counter adds phases (on average 9 instead of 7), because survivors no longer have to be local maxima; B then sends about 1700 more M1s and saves about 800 M2s, so it is about 9% more expensive than A+1+2. The counter buys the worst-case guarantee, which is what the paper optimises.

6 Points that are easy to misunderstand

- **Only M2 is limited, not M1.** An M1 always travels to the next active process, however far: that process needs it to compare. The counter only decides who becomes a candidate on the way; after 0 the M1 is a plain relay.
- **Waiting and active are different moments.** A virtual candidate becomes waiting when the M1 with counter 1 passes it, and active only later, when the M2 arrives. An active process stops being active as soon as it receives an M1: a bigger value makes it wait, a smaller one makes it passive and send M2.
- **Why a waiting process never forwards M2.** The M2 carries exactly the value it waits for (step B2 of the paper notes that $j = \max(v)$), so it takes the value; stopping there is what bounds the length of M2 by 2^p.
- **Who drops an M2.** Any passive process whose max is larger, including the next active process itself after it lost the comparison (in the example ring $5, 2, 6, 3, 1, 4, 8, 7$, P6 drops M2(4) in phase 0 because it holds 8).
- **Phases without synchronisation.** A phase is just the number carried by M1. Each process receives exactly one M1 per phase, and by FIFO the messages of different phases never mix on a link. In the code this also makes every count independent of the schedule: each process is deterministic and reads one FIFO link (tested by `TestScheduleDoesNotMatter`).
- **The constant $\frac{5}{3}$** in the bound for M2 is not tight: the series gives about $0.80\,n$.

7 How to run the code

Install Go 1.21 or newer (`https://go.dev/dl/`, or `brew install go` on macOS). The code has no other dependencies.

```
git clone https://github.com/RomanVolochai/distributed-systems-unidir-leader-finder-b.git
cd distributed-systems-unidir-leader-finder-b/code
go run .                              # random ring of 8 processes, full log
go run . -ids 5,2,6,3,1,4,8,7         # the ring from the talk (26 messages)
go run . -file examples/ring8.txt -delay 300ms     # follow it live
go run . -n 1000 -quiet [-algo A]     # summary only; -algo A: Algorithm A
go run . -n 64 -layout bitrev -quiet  # random, asc, desc or bitrev
go run . -bench [-maxn 4096]          # the scaling table above
go test                               # all checks, prints "ok"
```

Every run prints its seed; `-seed` repeats the same ids and the same order of steps. All flags are listed by `go run . -h` and in `README.md`.